SOFTWARE ENGINEERING

Ms. Archana A, Sowmya BP & Akshatha PR
Department of Computer Applications

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

System Modeling

- User requirements are written in natural language.
- One widely used technique is to document the system specification as a set of **system models**.
- These models are **graphical representations** that describe **business processes, the problem to be solved and the system that is to be developed**.
- Because of the graphical representations used, **models are often more understandable** than detailed natural language descriptions of the system requirements.

System Modeling

The most important aspect of a system model is **an abstraction** of the system being studied rather than an alternative representation of that system.

Different **types of system models** are based on different approaches to abstraction.

Examples of the types of system models that are created during the analysis process are:

1. **A data- flow model :** Data-flow models show how data is processed at different stages in the system.
2. **An architectural model:** Architectural models show the principle sub-systems that make up a system.

3. A classification model : Object **class/inheritance diagrams** show how entities have common characteristics.

4. A composition model : A **composition or aggregation** model shows how entities in the system are composed of other entities

5. A stimulus-response model: A stimulus-response model, or **state transition diagram**, shows how the system reacts to internal and external events.

Here are the main categories of system models normally construct for all software applications:

1. **Structural models** – **class diagrams**,
2. **Behavioural models** – **state diagrams**
3. **Interaction model-** use case diagram, **sequence diagram**

PES UNIVERSITY

1. **Structural models** –

- Structural models represent the **static structure** of a system. They focus on the **components** of the system, their **relationships**, and their **organization**.
- These models do not show the dynamic behavior of the system but rather its building blocks.

- **For example: Class Diagram:** Represents the classes, attributes, methods, and relationships (e.g., inheritance, association, aggregation, composition) between objects.

2. Behavioral models –

- Behavioral models represent the **dynamic behavior** of a system. They focus on how the system **responds to events**, **interacts with users**, and **changes over time**.

For example: **State Transition Diagram:** Represents the states of an object and how it transitions between states in response to events.

PES UNIVERSITY

3. Interaction model-

- **Interaction Models**, focus on the **interactions** between different components or objects in the system. They are a subset of behavioral models but emphasize the **communication** and **collaboration** between entities.

for example: **Sequence Diagram:** Shows the sequence of messages exchanged between objects over time.

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

1. Structural Models – class diagram

Archana A
Computer Applications

Structural Models - Introduction

What is Model?

A model is a **simplification of reality**, which provides the **blueprints** of a system.

We use models in almost everything that we do,
for example:

- When we get up in the morning and get ready to leave for work, we have a rough plan of the day ready in our mind – ***model of the day***.
- Similarly, an engineer prepare outline of building- ***blueprint***
- Authors prepare rough Table of Content(TOC) for book- ***contents of book***

Structural Models

Why to build Models ?

Models serves several **purpose:**

1. Testing a physical entity before building it.
2. Communication with customers
3. Visualisation
4. Reduction of complexity.

Structural Models

Unified Modeling Language:

- The UML is a **visual language** for **specifying, constructing and documenting** the artifacts of system.

- It is a standard language for writing **software blueprints**.

The building blocks of UML are:

- **Things (entities)**
- **Relationships**
- **Diagrams**

Structural Models

- **Entities are the elements of models**.

- All UML entities can be divided into 3 entities.
  They are:

1. **Structural entities :-** **nouns** -UML models, such as class, interface, abstract class etc.

2. Behavioral entities - **verbs** UML models, such as interaction, activity among the entities.

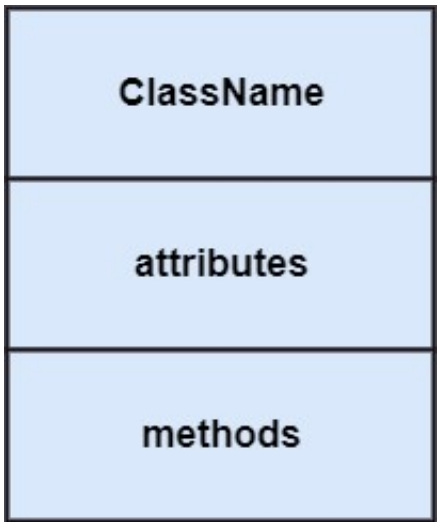
3. Summary entity - **note**, which can be added to the model to record specific information, very much like the **sticker**

Structural Models

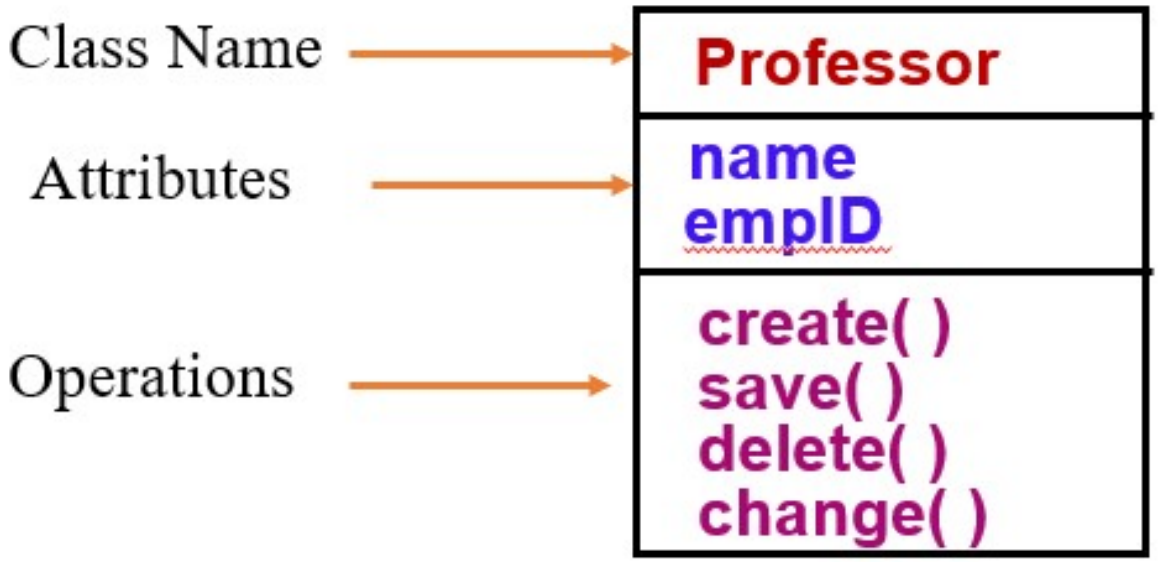
Class: a class is a set of objects that share the same attributes, operations, relationships and semantics

A class is represented using a **compartmented rectangle**. It comprised of three sections

- The **first section** contains the class name
- The **second section** shows the structure (attributes)
- The **third section** shows the behavior (operations or methods)

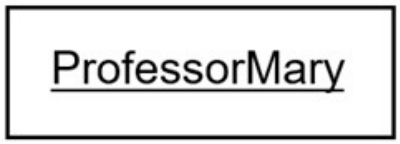


Class Example



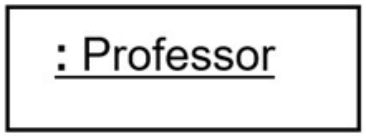
Representing Objects in UML

An object is represented as rectangles with **underlined names**



Object Name Only

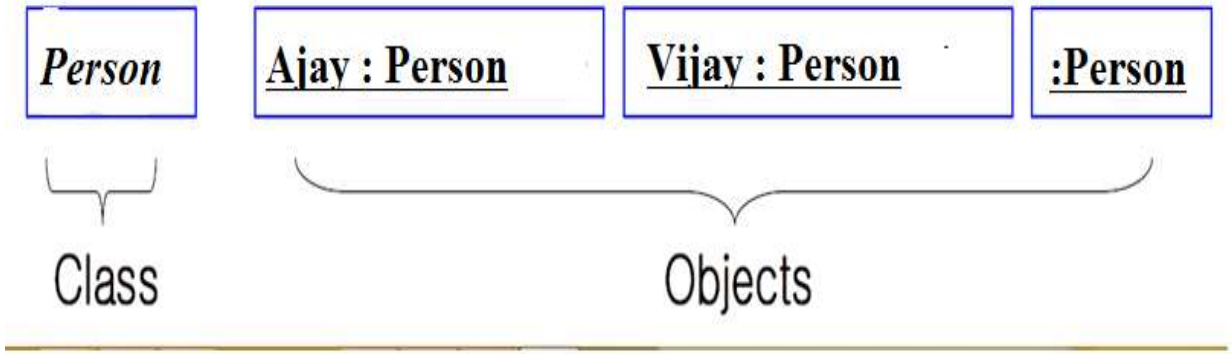
The other forms of object representations are:



Class Name Only

Object Name with its Class Name

For Example:



Note: classes & objects are the focus of class modeling

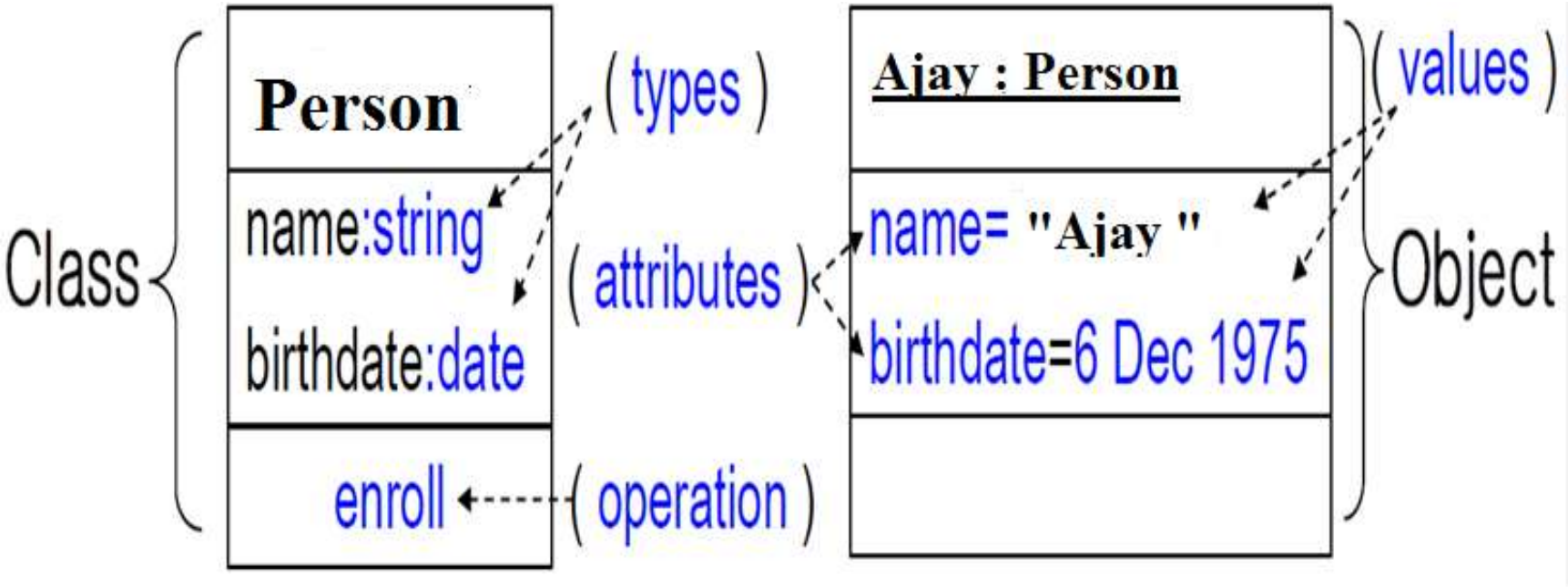
Values and Attributes

- **Value :** is a piece of data
- **Attribute :** value for each object. It is a named property of class.
- **Operations(Methods)**

Function or procedure that may be applied to or by objects in a class

- All objects in a class share the same operations

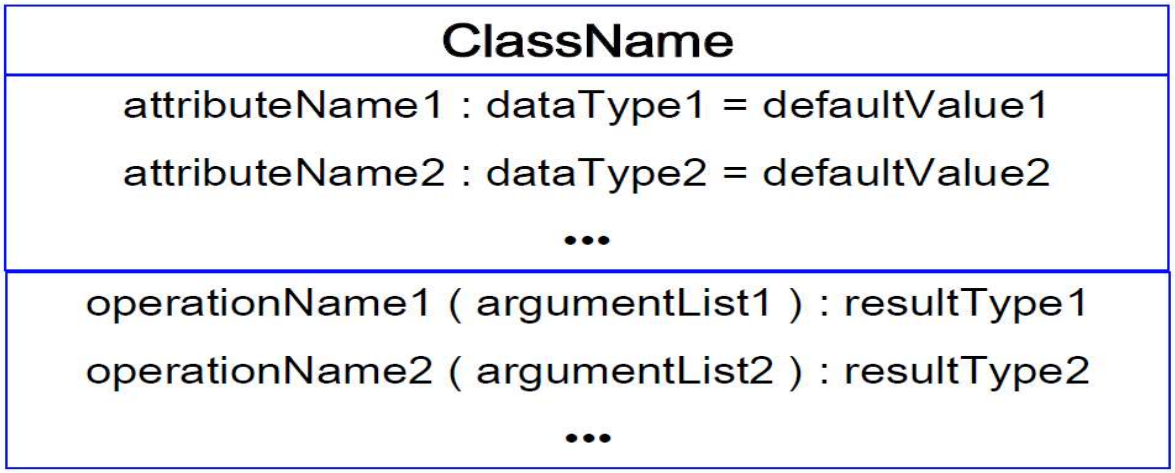
For Example:


Class
Person
name:string
birthdate:date
enroll
(types)
(attributes)
(operation)
Ajay : Person
name= "Ajay "
birthdate=6 Dec 1975
(values)
Object

Summary of Notation for Class

- A box represents a class and may have as many as three compartments. It contain from top to bottom: class name, list of attributes, and list of operations.

- The attribute and operation compartments are optional and missing attribute compartment means that are unspecified.



SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

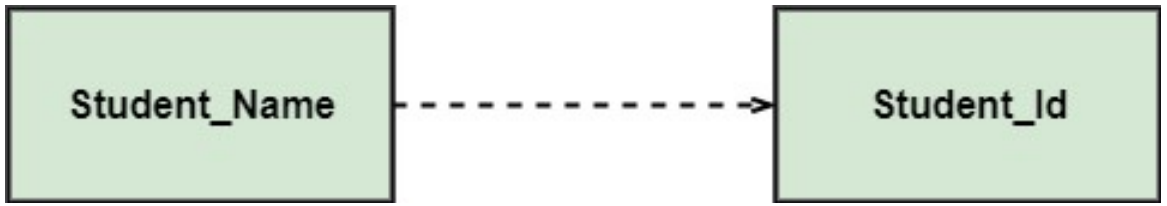
Structural Models

2. Relationships:

Type of relationship	UML syntax Source – Target	Semantics
Dependency		Source element depends on the target element and change last may affect the first.
Association		Description of the set of relationships between objects.
Aggregation		The target element is a part of the source element.
Composition		Strict (more limited) form of aggregation/
Generalization		The source element is a specialization of a more generalized the target element, and can replace him.

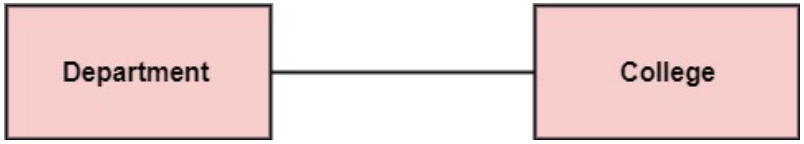
1. **Dependency:** A dependency is a semantic relationship between two or more classes where a change in one class cause changes in another class. It forms a weaker relationship.

Example-- Student_Name is dependent on the Student_Id.



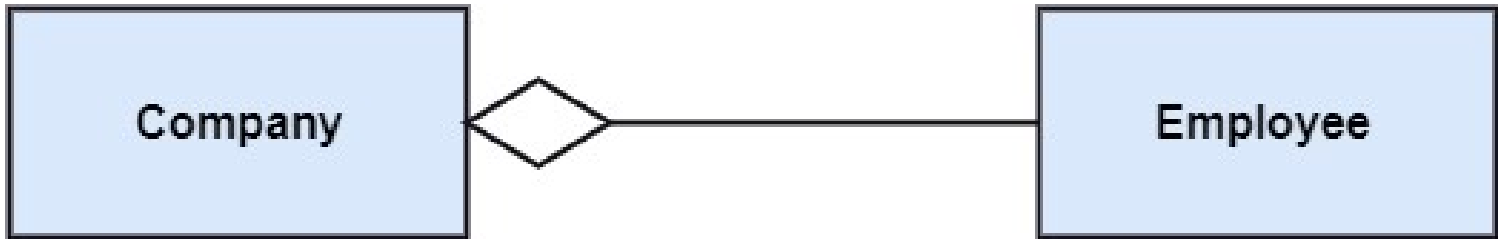
2.Association: It describes a **static or physical connection** between two or more objects. It depicts how many objects are there in the relationship.

For example, a department is associated with the college.



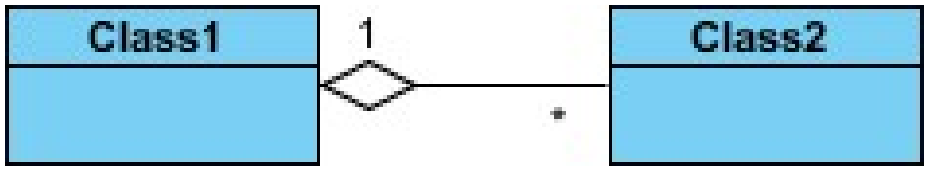
3. Aggregation: An aggregation is a special form of association,. It is more specific than association. It defines a **part-whole or part-of relationship**.

- Example: The company encompasses a number of employees, and even if one employee resigns, the company still exists.



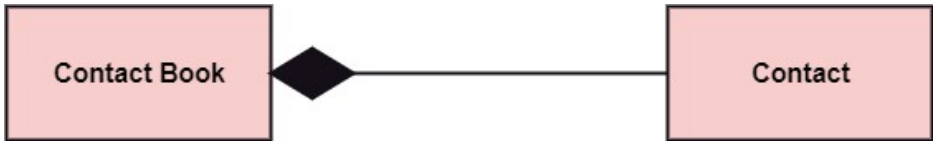
A special type of association. It represents a "part of" relationship.

- Notation: A solid line with an **unfilled diamond** at the association end connected between two composite classes



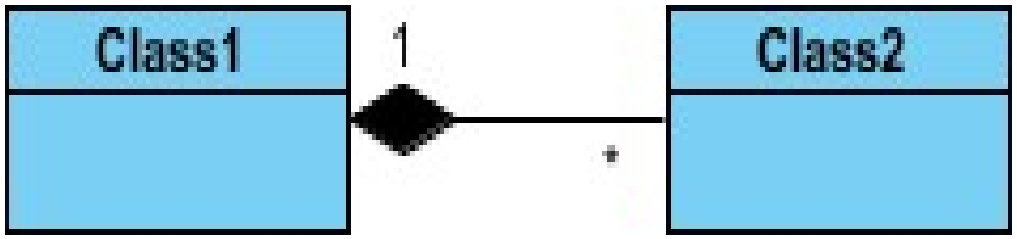
4. Composition: The composition is a subset of aggregation. It portrays the dependency between the container class and composed class, which means if one part is deleted, then the other part also gets discarded. It represents a **whole-part relationship.**

Example--A contact book consists of multiple contacts, and if you delete the contact book, all the contacts will be lost.



Composition: A special type of aggregation where parts are destroyed when the whole is destroyed.

- Objects of Class2 live and die within Class1.
- Class2 cannot stand by itself.
- A solid line with a filled diamond at the association connected to the class of composite

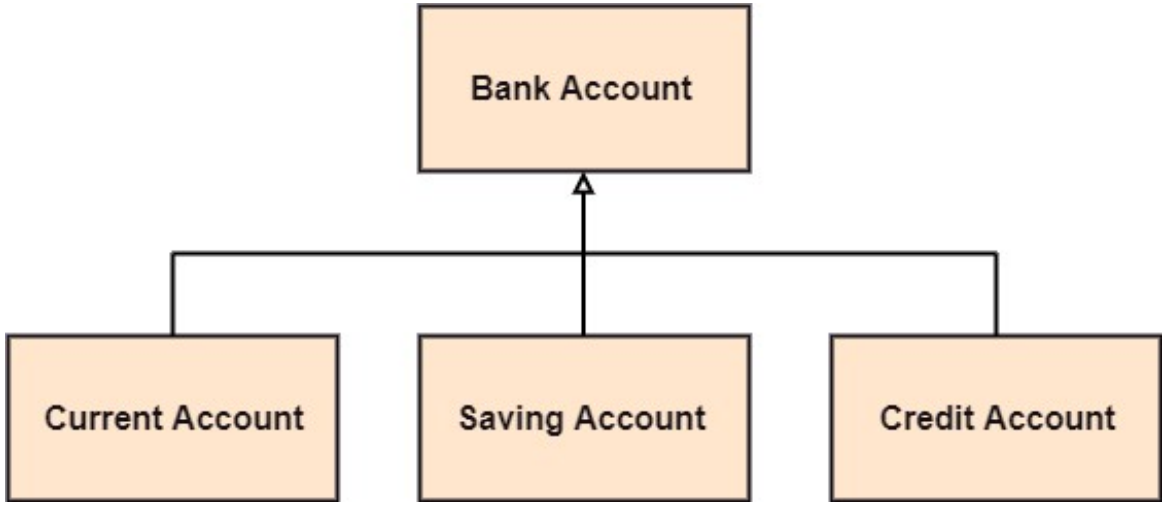


5. Inheritance (or Generalization):

✓ Represents an "**is-a**" relationship.

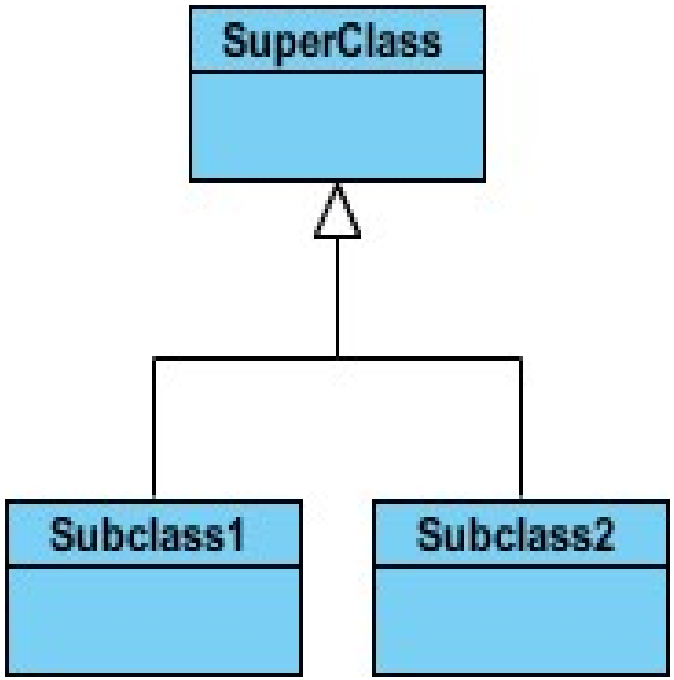
A generalization is a relationship between a parent class (superclass) and a child class (subclass). In this, the child class is **inherited** **f**rom the parent class.

Example-- The Current Account, Saving Account, and Credit Account are the generalized form of Bank Account.

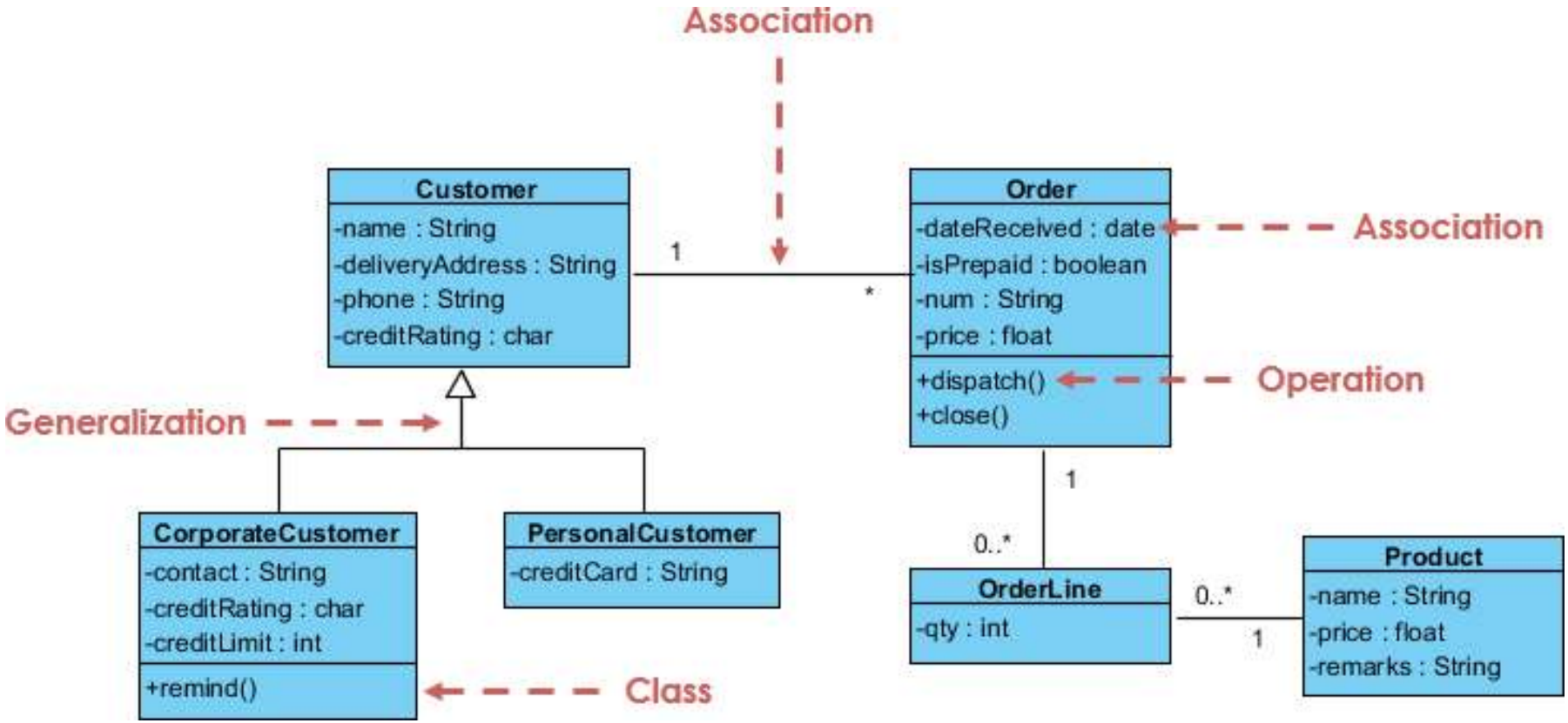


Inheritance (or Generalization):

- SubClass1 and SubClass2 are specializations of Super Class.
- A solid line with a hollow arrowhead that point from the child to the parent class



Example – Sales Order system



Multiplicity

- Multiplicity constrains the number of related objects.
- It specifies the **number of instances of one class that may relate to a single instance of an associated class**.
- Assign a multiplicity to an **association end**

Multiplicity…

UML diagrams specifies the multiplicity with an interval such as

"1" → exactly one

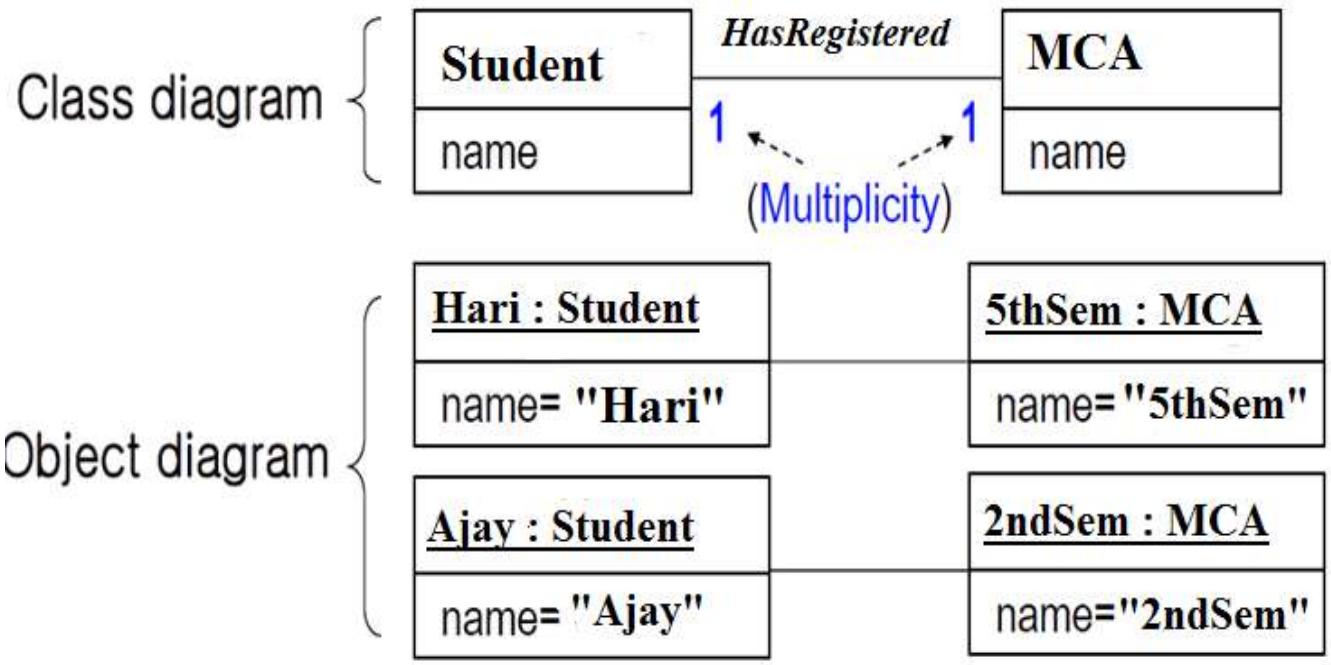
"1..*" → one or more

"3.5" → three to five (inclusive)

" *.* " → many (i.e, zero or more)

Multiplicity Example…

For Example:



SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

Class Diagram

1. **Construct a class model for Library application. Given** Librarian, LibStaff, Members, membership_id, m_address, publication, BookSearch, Book_issue, Book_Return, penalty, Catalogue search,Purchase book, Place order, IEEE Conference Paper, e-courses, e-journals, RegisterCourse, ISBN

Class Diagram

Scenario:

A library manages books and members. Each book has a title, author, ISBN, and publication year. Members can borrow multiple books. Each borrowed book has an issue date and return date. The librarian manages book records.

Question:

Identify the classes, attributes, and relationships required to create a UML Class Diagram.

Class Diagram

Solution:

Classes

Book (ISBN, title, author, publicationYear)
Member (memberID, name, email)
Loan (issueDate, returnDate)
Librarian (librarianID, name)

Relationships

Member **borrows** Book through Loan
One Member → many Loans (1..*)
One Book → many Loans (1..*)
Librarian **manages** Books

2. Online Shopping System

Scenario:
An online shopping system allows customers to browse products and place orders. Each product has a product ID, name, and price. Customers can add multiple products to their cart and place an order. Each order contains multiple order items.

Question:
Identify the classes and relationships for the UML Class Diagram.

2. Online Shopping System…
Solution:

Classes
Customer (customerID, name, email)
Product (productID, name, price)
Order (orderID, orderDate, totalAmount)
OrderItem (quantity)

Relationships
Customer **places** Order (1..*)
Order **contains** OrderItem (1..*)
OrderItem **refers to** Product (1)

PES UNIVERSITY

3. Hospital Management System

Scenario:
A hospital maintains records of doctors and patients. A patient can consult multiple doctors, and doctors can treat multiple patients. Each consultation includes diagnosis and treatment details.

Question:
Design the UML Class Diagram by identifying classes and associations.

3. Hospital Management System

Solution:

Classes

Doctor (doctorID, name, specialization)
Patient (patientID, name, age)
Consultation (consultationDate, diagnosis, treatment)

Relationships

Doctor ↔ Patient (**many-to-many**) through Consultation
One Doctor → many Consultations
One Patient → many Consultations

4) Draw Class Diagram using the below given classes and relationships:

Classes:

Customer, Product, Category, Cart, Order

Relationships:

Customer has a composition relationship with Cart.

Cart has an association relationship with Product.

Product has an association relationship with Category.

Customer has a composition relationship with Order.

4. University Course Registration System

Scenario:
A university offers multiple courses. Students enroll in courses. Each course is taught by a professor. A student can enroll in many courses.

Question:
Identify the classes and relationships.

4. University Course Registration System

Solution:

Classes

Student (studentID, name, program)
Course (courseID, courseName, credits)
Professor (professorID, name, department)
Enrollment (enrollDate, grade)

Relationships

Student ↔ Course (many-to-many) via Enrollment
Professor **teaches** Course (1..*)

5. Ride Booking System

Scenario:
A ride booking application allows users to request rides. Drivers accept ride requests. Each ride contains pickup location, drop location, and fare.

Question:
Create classes and relationships for a UML class diagram.

5. Ride Booking System

Solution:

Classes

User (userID, name, phone)
Driver (driverID, name, vehicleNumber)
Ride (rideID, pickupLocation, dropLocation, fare)

Relationships

User **requests** Ride (1..*)
Driver **accepts** Ride (1..*)
Each Ride has **one User and one Driver**

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

2. Behavioral Models – State transition diagram

Archana A
Computer Applications

State Model

- **State model** express **behavioral aspects** of a system.
- Multiple state diagram forms the state model.
- The various state in an object's life cycle are depicted using state transition diagram
- To construct the state transition diagram we need to know the events that causes change of an object's state to take place.
- Hence we need **Events and State** to construct state diagram.

- Events represent **points in `time‘**;

- states represent **interval of time**, ie., the abstract state of an object (or collection of objects).
- Consider the switch object as an example:

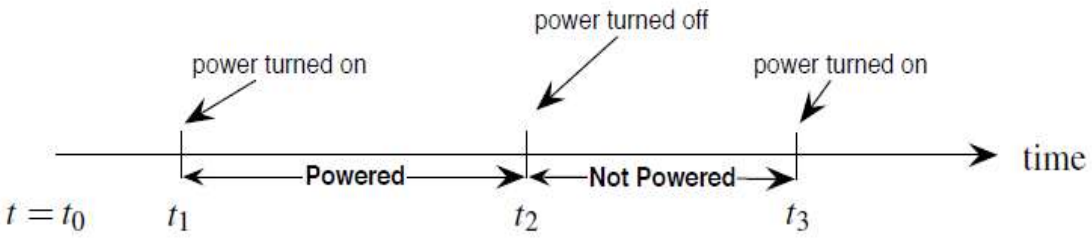


Figure: Power up, power down

Event Vs State…

- The objects in a class have finite number of possible states. Each object can only be in **one state at time.**
- Object pass through one or more states during the life time.
- A state specifies the response of an object to input events

Event Vs State…

- An events are ignored in a state, except those for which behavior is explicitly prescribed.
- Eg: if a digit is dialed in state Dial tone, the phone line drops the Dial tone & enters state **dialing**

Transitions and Conditions

- A **transition** is an **instantaneous change** from **one state to another**.

- UML notation for transition is:
  A **line** drawn from the **origin state** to the **target state**.

Transitions and Conditions

- A transition is said to fire upon the change from the **source state** to the **target state**.
    - Ex: when a called phone is answered, the phone line transitions from the Ringing state to Connected State.

Transitions and Conditions

- A **guard condition** is a Boolean expression that must be true in order for a transition to occur.

- The syntax for guard and event along a transition is :

event[guard]

Consider the following example:

Here identify events and states

"When you go out in the morning, if the temperature is below freezing, then put on your gloves"

When you go out in the morning (Event),

if the temperature is below freezing (guard condition),

then put on your gloves(action/event).

Statement2: "When a customer makes a purchase online and the payment is successful, send a confirmation email.“

Statement3: "If the user enters the wrong password three times consecutively, lock the account.“

Statement4: "When a sensor detects motion in a room, turn on the lights. If there is no motion for 10 minutes, turn off the lights."

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

State Diagram

Ms. Archana A, Sowmya BP & Akshatha PR
Department of Computer Applications

- A state diagram is a **graph** whose **nodes are states** and whose **directed, annotated arcs** are **transitions** between states.

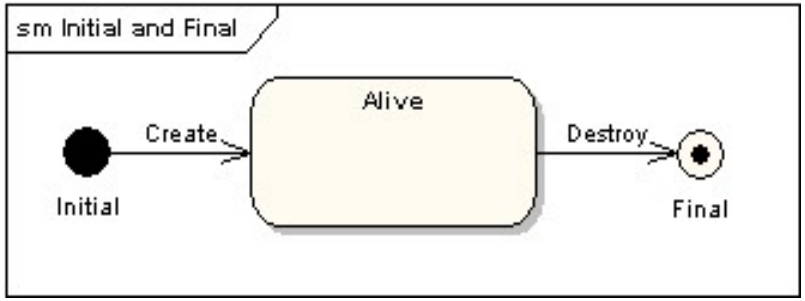
Some constraints behind the structure of a state diagram:

- State names must be unique within the scope of a state diagram
- All objects in a class execute the state diagram for that class, which models their common behaviour

State diagram – Components used

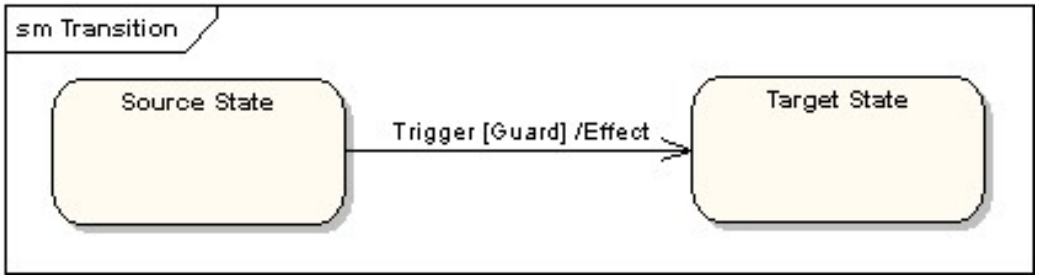
1. Initial state – **Filled circle** represent the initial state of a System

2. Final state – Use a **filled circle within a circle notation** to represent the final state in a state machine diagram.

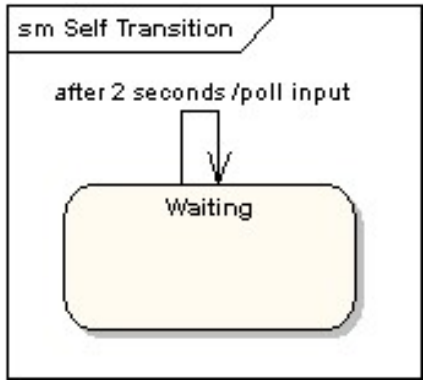


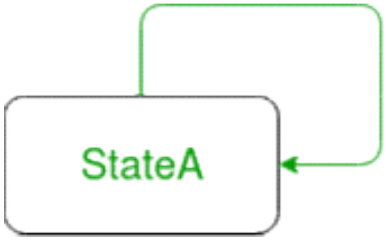
3. State – Rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

4. Transition – Solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.



5. Self transition – Solid arrow pointing back to the **state itself** to represent a self transition. There might be scenarios when the state of the object does not change upon the occurrence of an event. use self transitions to represent such cases.





State Diagram

- The **state model** consists of multiple state diagrams, one state diagram for each class.

- State diagram can represent
  - **Continuous loop** or
  - **One-shot life cycle**.

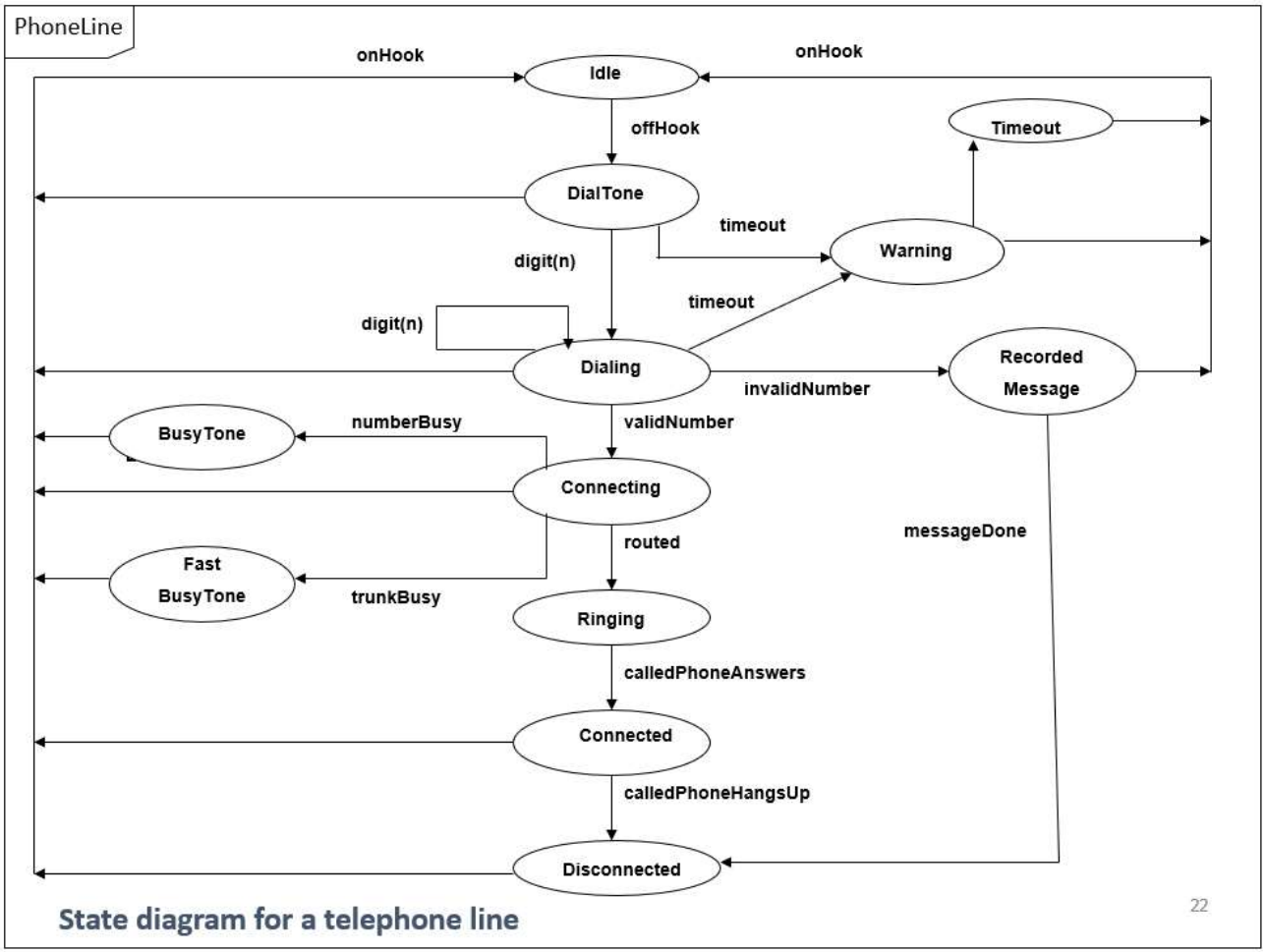
Example – state diagram for Telephone Line

Consider the class for telephone line with following activities and states:

As a start of a call, the telephone line is **idle**. When the phone receiver is picked from hook, it gives a **dial tone** and can accept the **dialing of digits**. If after getting dial tone, if the user doesn't dial number within time interval then **time out** occurs and phone line gets idle. After dialing a number, if the number is invalid then some **recorded message** is played. Upon entry of a valid number, the phone system tries to **connect a call** & routes it to proper destination. If the called person **answers the phone**, the conversation can occur. When called person hangs up, the **phone disconnects** and goes to idle state.

SOFTWARE ENGINEERING

State Diagram – Continuous loop example



State diagram for a telephone line

SOFTWARE ENGINEERING

One-Shot State Diagram

Ms. Archana A, Sowmya BP & Akshatha PR
Department of Computer Applications

One shot State Diagram

- One shot diagrams **represents objects with finite lives** and have **initial** & **final** states.
- A one shot diagram has a **start** - depicted as a **solid circle** - corresponds to **object creation**
- A **final state** is depicted with a **bulls eye** - corresponds to **object destruction**.

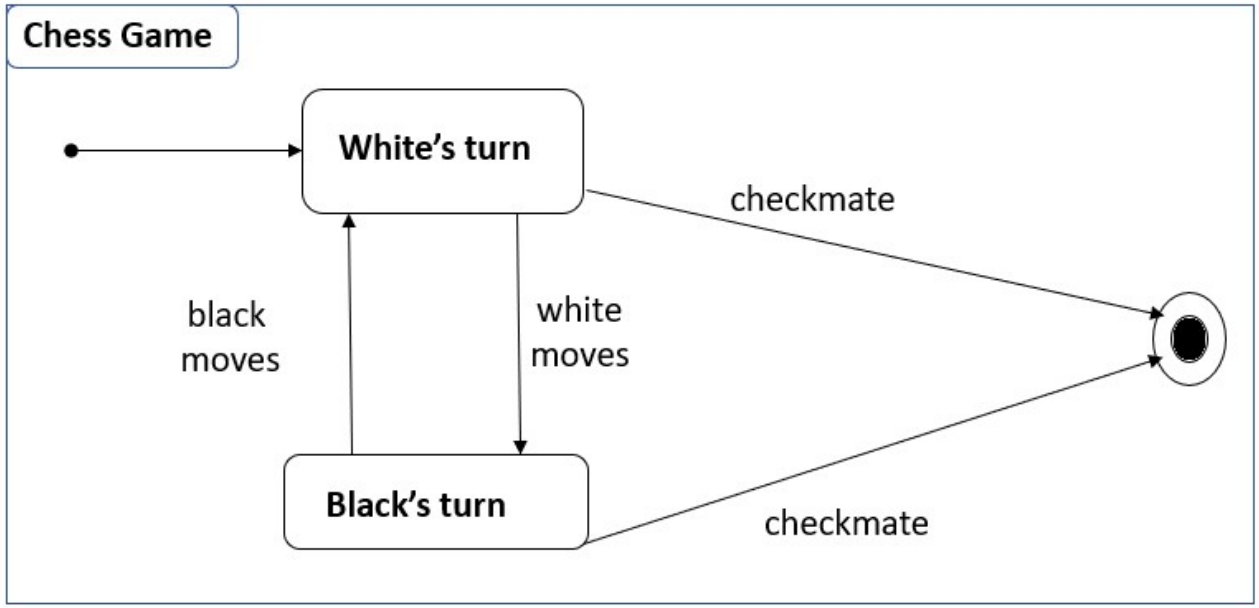
One shot State Diagram

- One-shot state diagrams represent objects with **finite lives** and have **initial and final states.**
- The **initial state** is entered on **creation** of an object; entry of the final state implies **destruction** of the object.
- The next figure shows a simplified life cycle of a chess game with a default initial state (solid circle) and a default final state (bull's eye)

One shot State Diagram

For example:

- The figure shows a **simplified life cycle** of a **chess game** with a default initial state (solid circle) and a default final state (bull's eye)



SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

Draw a continuous loop state diagram for ATM object

One Shot State Diagram

For the Given scenario, identify the events that causes the change of state to takes place and draw a one-shot state transition diagram

1. Account Object

- **An account starts in the "Active" state.**
- **If there are suspicious activities or violation of terms, the account can be "Blocked.“ From the "Blocked" state, the account may be further escalated to the "Suspended" state for severe violations.**
- **If issues are resolved or penalties lifted, the account can be restored to the "Active" state from either "Blocked" or "Suspended.“**
- **Account no longer used it reaches to “closed” state**

For the Given scenario, identify the events that causes the change of state to takes place and draw a one-shot state transition diagram

2. Door Lock object

- **The door lock starts in the "Locked" state.**
- **It can be unlocked either by using a "Key" or entering the correct code on the "Keypad."**
- **If unlocked with the "Key," it transitions to the "Unlocked" state.**
- **Similarly, if unlocked with the "Keypad," it also transitions to the "Unlocked" state.**

For the Given scenario, identify the events that causes the change of state to takes place and draw a state transition diagram

3. Vending Machine

- **Design a state diagram for a simple vending machine that sells two products: "Chips" and "Soda." The machine has three states: "Idle," "Chips Selected," and "Soda Selected." Once a product is dispensed, it goes back to the "Idle" state.**

For the Given scenario, identify the events that causes the change of state to takes place and draw a state transition diagram

4. Music Player

- **Create a state diagram for a simple music player with three states: "Stopped," "Playing," and "Paused." The player can transition from Stopped to Playing, from Playing to Paused, and from Paused back to Playing.**

For the Given scenario, identify the events that causes the change of state to takes place and draw a state transition diagram

5) Online Order Object

The order starts in the **"Created"** state.
When the payment is successful, it moves to the **"Confirmed"** state.
The order is then processed and moves to the **"Shipped"** state.
After delivery, it transitions to the **"Delivered"** state.
At any point before shipping, the order can be moved to the **"Cancelled"** state.

For the Given scenario, identify the events that causes the change of state to takes place and draw a one-shot state transition diagram

6) Elevator Object

Scenario:

An elevator waits idle. When a user presses a button, it moves up or down, stops at the requested floor, and returns to idle..

PES UNIVERSITY

6) Elevator Object
Solution:

- The elevator starts in the **"Idle"** state.
- When a request is received, it moves to either **"Moving Up"** or **"Moving Down"** state.
- Upon reaching the destination floor, it transitions to the **"Door Open"** state.
- After some time, it returns to the **"Idle"** state.

.

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

3. Interaction Model – Sequence diagram

Archana A
Computer Applications

- The Sequence model **elaborates the theme of use cases.**
- There are two kind of sequence models:-
  - **Scenarios.**
  - **Sequence diagrams** more structured format.

1. Scenario:

- A scenario is a **sequence of events** that occurs during **one** particular execution of a system, such as for a use case.
- A scenario can be displayed as a list of text statements

SOFTWARE ENGINEERING

Scenario for the session with Online Stock Broker

PES UNIVERSITY

John Doe logs in.
System establishes secure communications.
System displays portfolio information.
John Doe enters a buy order for 100 shares of GE at the market price.
System verifies sufficient funds for purchase.
System displays confirmation screen with estimated cost.
John Doe confirms purchase.
System places order on securities exchange.
System displays transaction tracking number.
John Doe logs out.
System establishes insecure communication.
System displays good-bye screen.
Securities exchange reports results of trade.

Scenario

- At early stages of development, Scenarios are expressed at a high level, later stages we can show exact messages.

1. First step of writing a scenario is to identify the objects exchanging messages.
2. Determine the sender and receiver of each message and sequence of messages.
3. Add activities for internal computations.

PES UNIVERSITY

2. Sequence Diagram:

- A sequence diagram shows the **participants** in an interaction and the **sequence of messages** among them.

- The components used to construct sequence diagram are

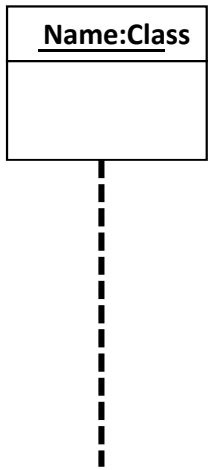
1. Object with lifeline
2. Messages exchanged among object

Components of Sequence diagram

1. Object:

- Objects are represented by rectangles and **name of the objects are underlined**

- Object life line are denoted as **dashed lines**. They are used to model the existence of objects over time



Sequence diagram…

- Each actor as well as the system is represented by a vertical line called lifeline and each message by a horizontal arrow from sender to the receiver.

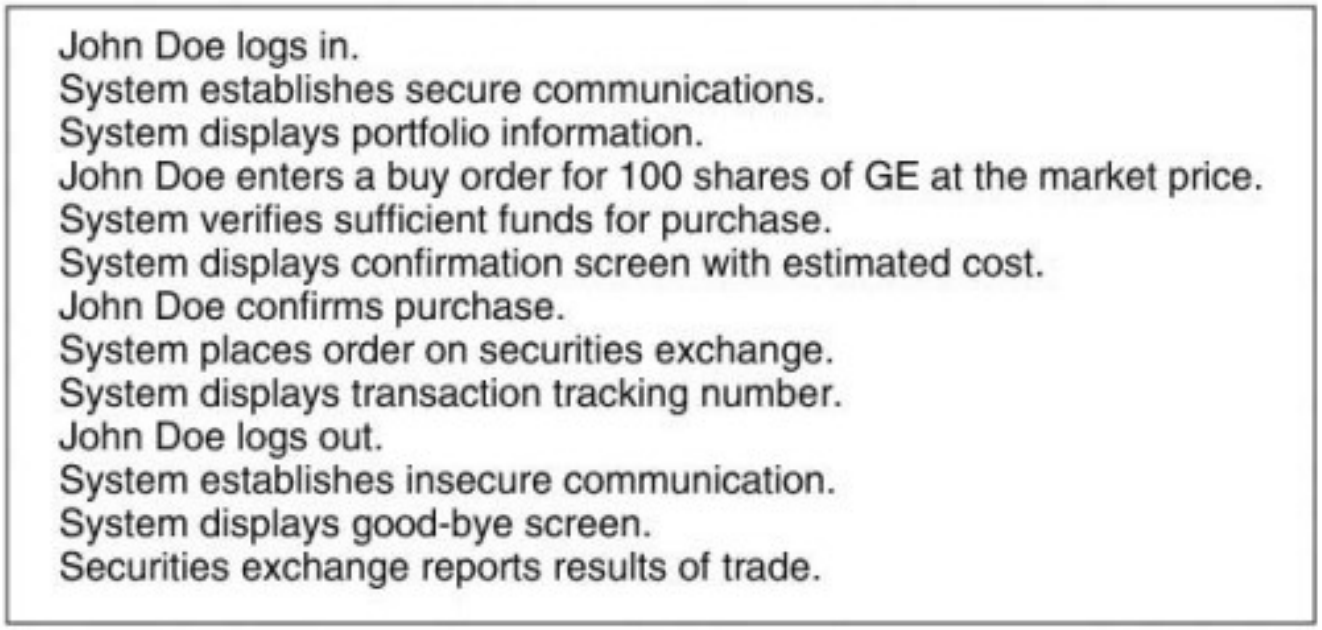
- Each use case requires one or more sequence diagram to describe its behavior.

- For large scale interactions containing many independent tasks, we can draw separate sequence diagram for each task.

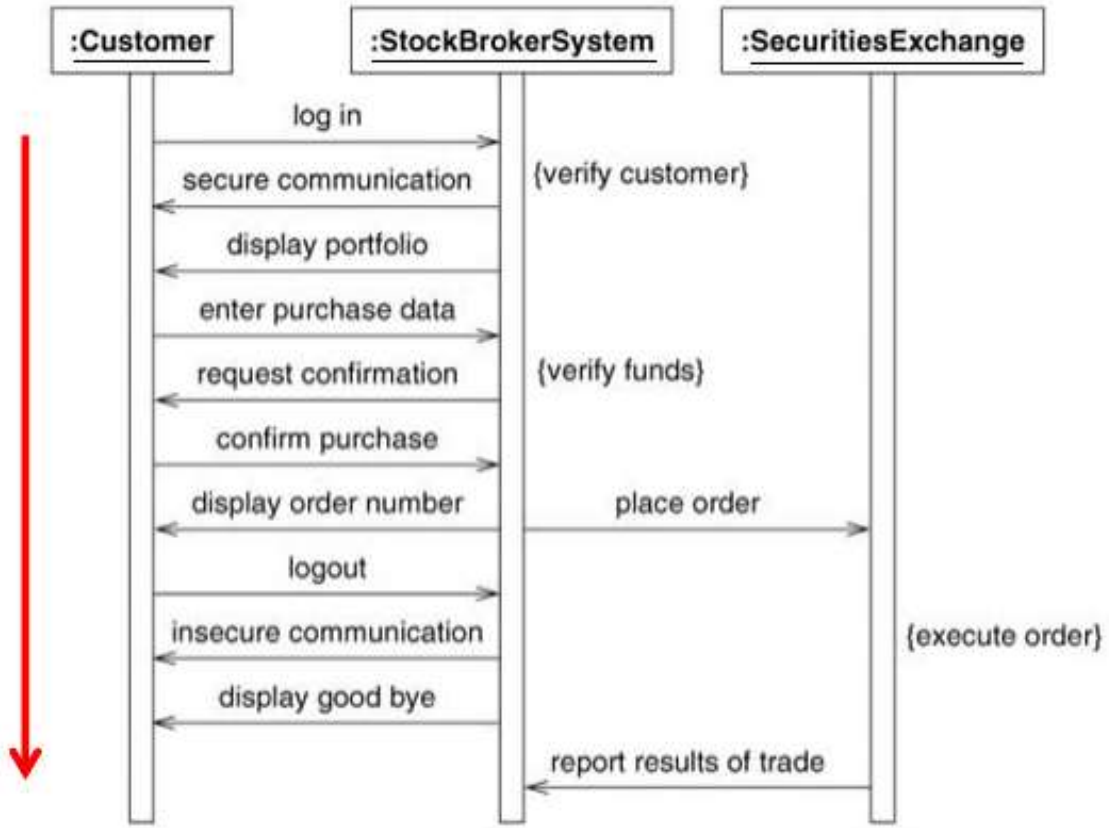
Eg:-

- Sequence diagram for a stock purchase
- Sequence diagram for a stock quote
- Sequence diagram for a stock purchased **that fails**.

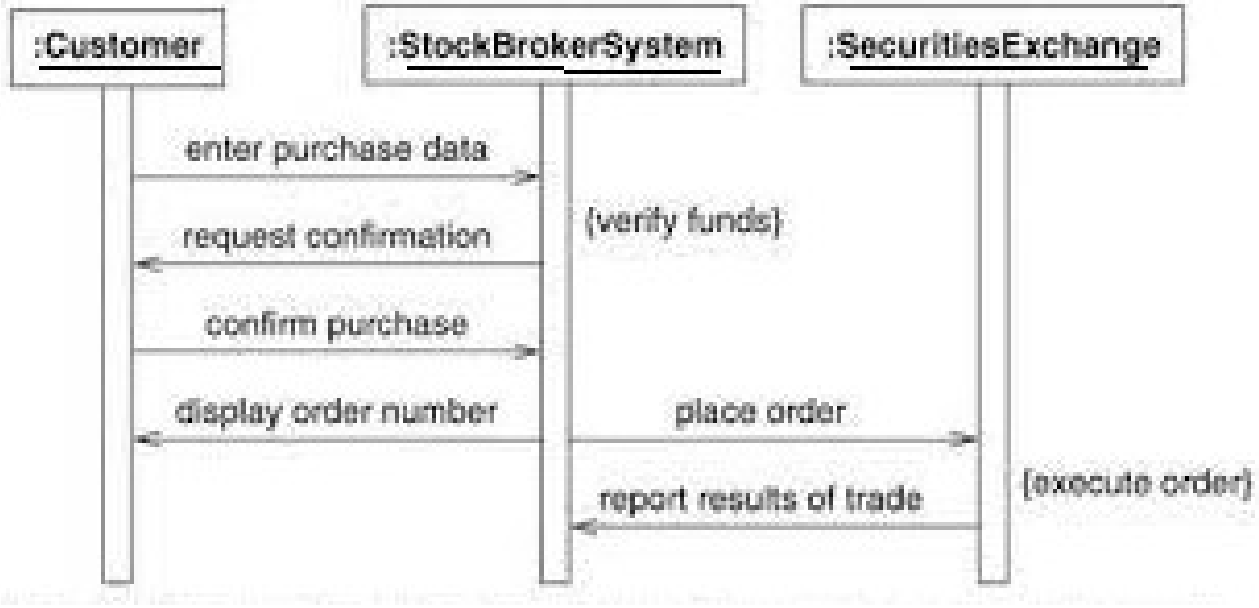
Scenario with sequence diagram example



Scenario with sequence diagram example



Sequence diagram – Stock purchase



Sequence diagram –stock quote

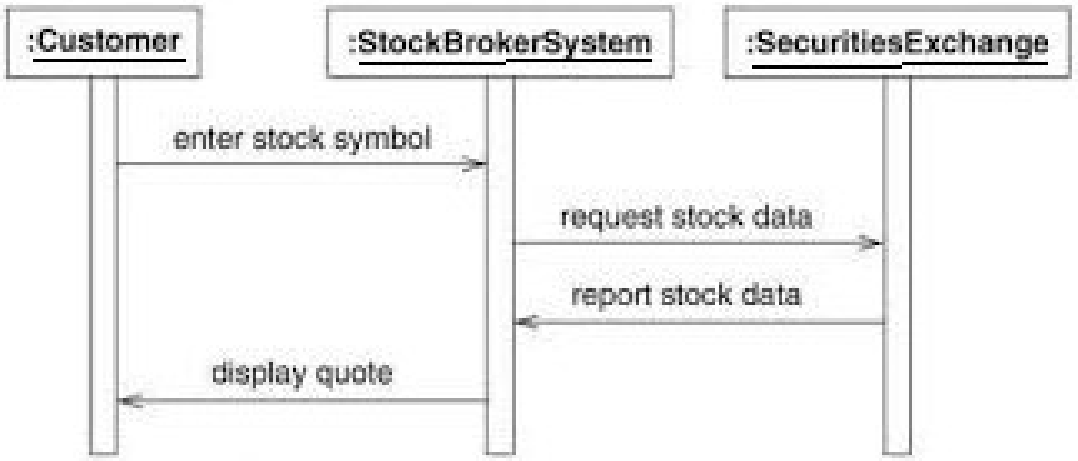


Figure Sequence diagram for a stock quote.

Scenario with sequence for stock purchase that fails

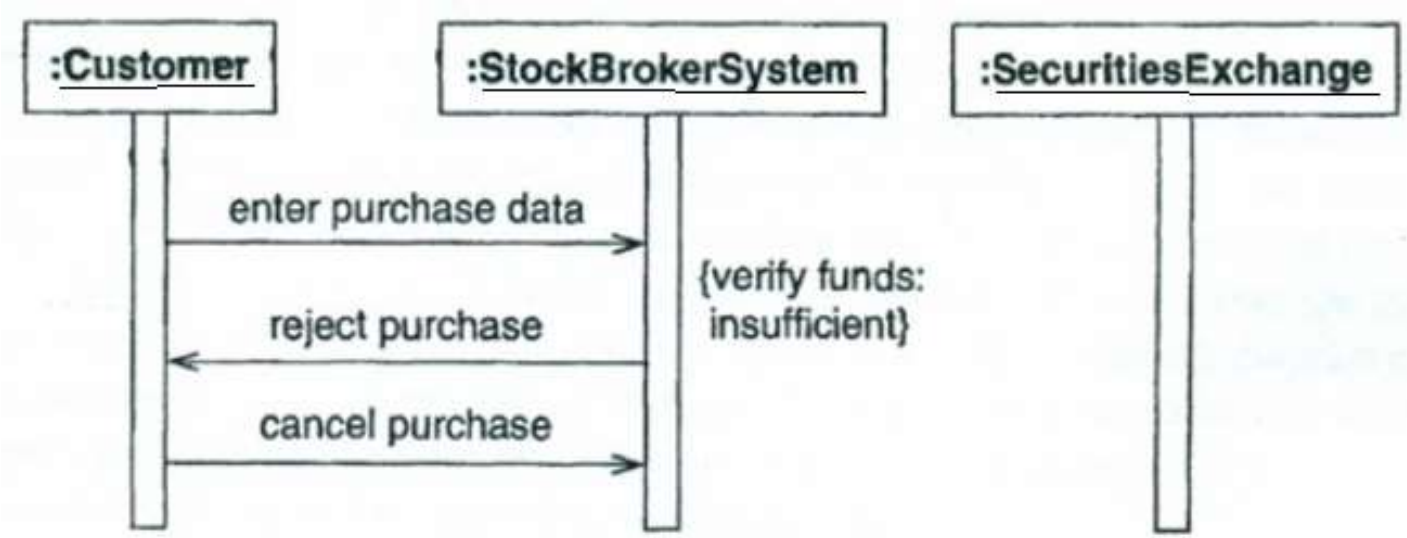


Fig Sequence diagram for a stock purchase that fails

2. Messages:

- In message **square brackets** containing **a guard conditions**. This is a Boolean condition that must be satisfied to enable the message to be sent.

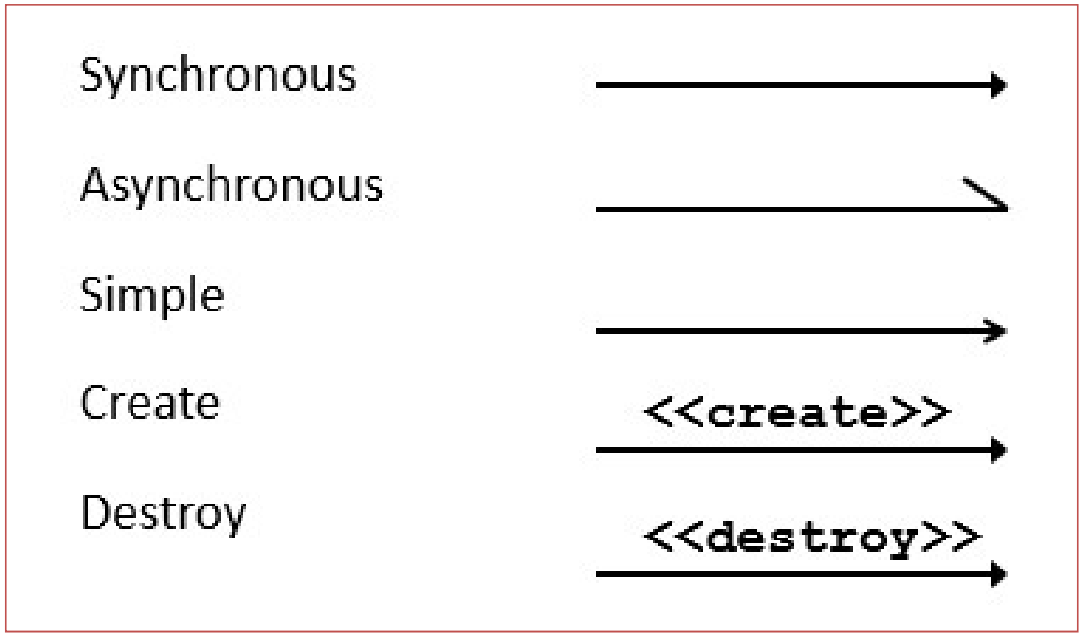
- May have an **asterisk followed by square brackets** containing an **iteration specification**. This specifies the number of times the message is sent.

Components of sequence diagram…

- May have **return list** consisting of a **comma -separated** list of names that designate the values of returned by the operation.
- Must have **a name or identifier** string that represents the message.
- May have **parentheses** containing an **argument list** consisting of a comma separated list of actual parameters passed to a method.

Messages…

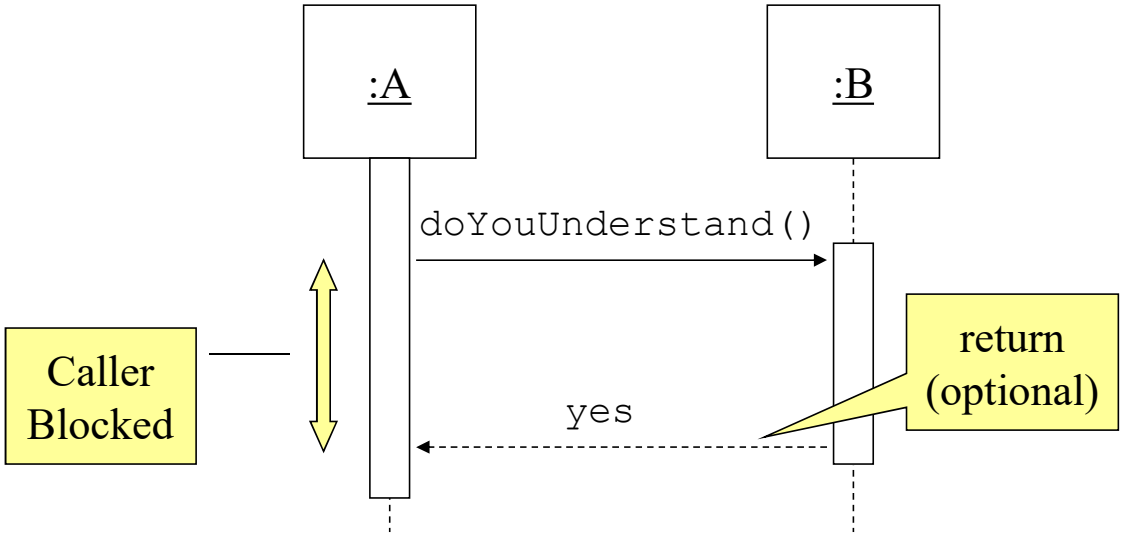
Types of Messages



Synchronous Messages:

Nested flow of control, typically implemented as an operation call.

The routine that handles the message is completed before the caller resumes execution.



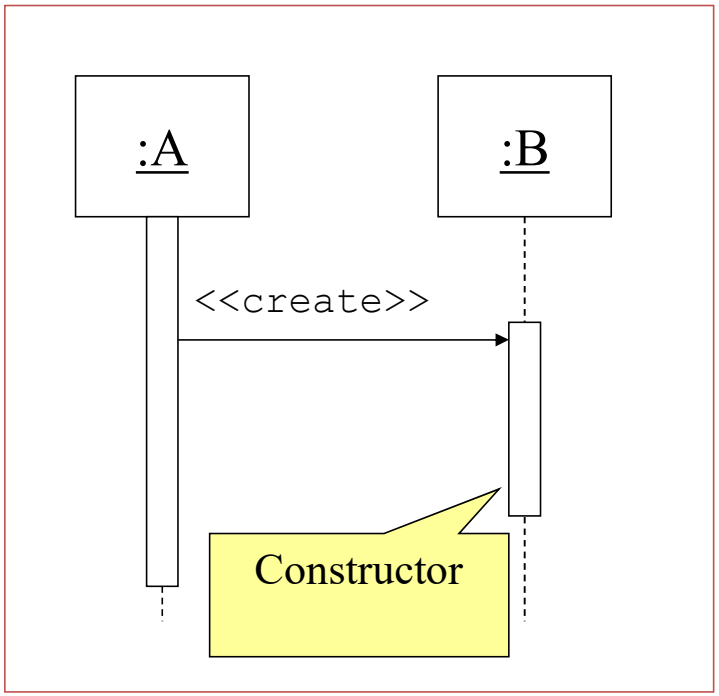
Messages…

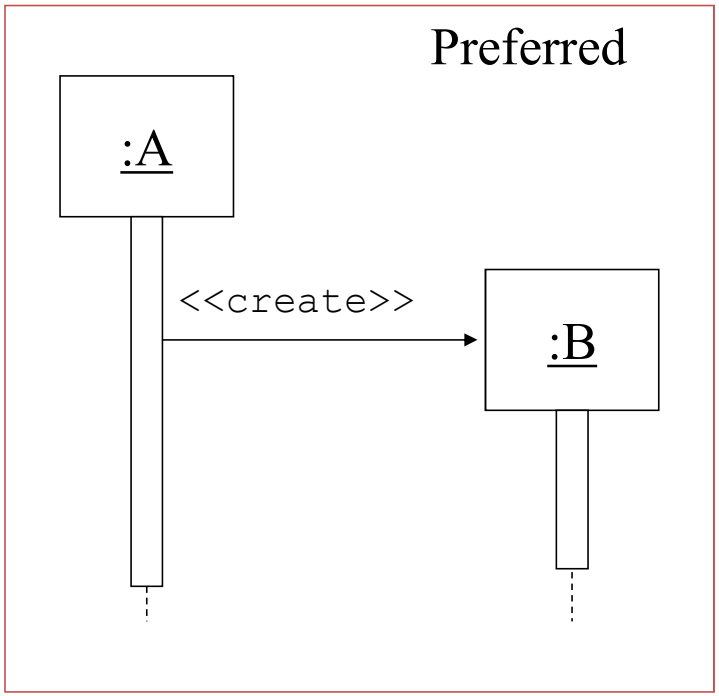
Optionally indicated using a dashed arrow with a label indicating the return value.

- Don’t model a return value when it is obvious what is being returned, e.g. getTotal()
- Model a return value only when you need to refer to it elsewhere, e.g. as a parameter passed in another message.
- Prefer modeling return values as part of a method invocation, e.g. ok = isValid()

Messages…

- **Object Creation:**
- An object may create another object via a **<<create>>** message.

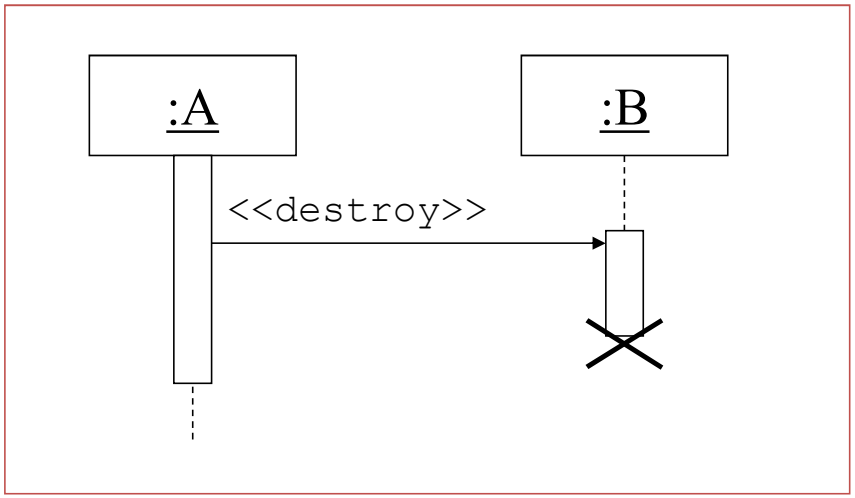




Object Destruction:

An object may destroy another object via a <<destroy>> message.

- An object may destroy itself.
- Avoid modeling object destruction unless memory management is critical.



SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

3. Interaction Model – Sequence diagram

Archana A
Computer Applications

Hands-on: Sequence diagram

For the given scenario, construct sequence diagram

1. ATM Withdrawal:

Objects: Customer, ATM Machine, Bank Server.

Interactions: Customer inserts card, enters PIN, validate pin, selects withdrawal amount, account balance is verified, cash is dispensed, transaction is recorded.

SOFTWARE ENGINEERING

Hands-on: Sequence diagram

For the given scenario, construct sequence diagram

2. Online Purchase Transaction:

Objects: Customer, Shopping Cart, Payment Gateway, Inventory System

Scenario: A customer purchases an item online.

Interactions:

- Customer adds items to the shopping cart.
- Shopping Cart updates the inventory.
- Customer initiates the checkout process.
- Payment Gateway processes the payment.
- Inventory System updates the stock.

Hands-on: Sequence diagram

For the given scenario, construct sequence diagram

3. Chat Application:

Objects: User, Chat Application Server, Database

Scenario: A user sends a message in a chat application.

Sequence of interactions:

- User types and sends a message.
- Chat Application Server receives the message.
- Server stores the message in the database.
- Server notifies the recipient of the new message.
- Recipient retrieves the message from the database.

SOFTWARE ENGINEERING

Hands-on: Sequence diagram

For the given scenario, construct sequence diagram

4. Online Hotel Room Booking System:

Objects: Customer, Hotel Booking System, Database, Payment Gateway.

Interactions: Customer searches for rooms, selects dates, enters personal information, availability is checked, price is displayed, customer confirms booking, payment is processed, confirmation email is sent.

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

Activity Models

Archana A
Department of Computer Applications

Activity Models

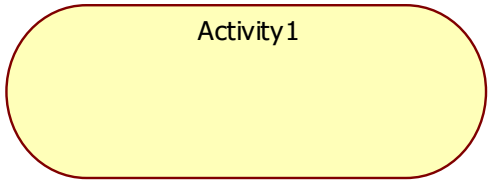
- Activity models are expressed through **Activity diagram**.
- An activity diagram shows **the sequence of steps that make up a complex process, such as algorithm or work flow**.
- It shows **flow of control** similar to sequence diagrams, but **focuses on operations rather than on objects.**
- Activity diagrams are most useful during the **early stages of designing algorithms and work flow.**

Components used to construct activity diagrams are:

- **Elongated oval** - shows activities and
- **arrows** - show their sequencing.
- **Diamond** - shows a decision point and
- **heavy bar** - shows splitting or merging of concurrent thread.

Activity Diagram components

1. **Activities -** represented as elongated oval



The steps of an activity diagram are operations, specifically activities from the state model.

Completion of activity is a completion of event and usually indicate that the next activity can be started.

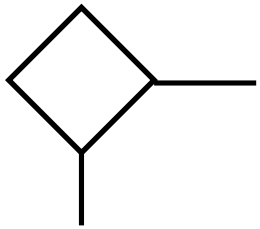
Unlabelled arrow ⟶ shows first activity must complete before the second activity can began.

2. Branch:

- If there is more than **one successor to an activity**, each arrow maybe labeled with a **condition in square** bracket.eg:- [failure] [success].

- All **subsequent conditions are tested** when an activity completes. If one condition is satisfied, its arrow indicates next activity to perform.

- As a notational convenience, a **diamond shows a branch into multiple successors.**



3. Initiation:

A **solid circle with an out going arrow** shows the starting point of an activity diagram.

4. Termination:

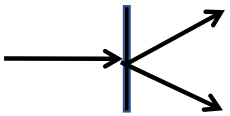
A **bulls eye**(a solid circle surrounded by a hollow circle) shows the **termination point**, this symbol has only **incoming arrow**.

5. Concurrent Activities:

Organizations and computer systems can perform more than one activity at a time.

Eg:- One activity may be followed by another activity(sequential control), then split into several concurrent activities (a fork of control) and finally combined into a single activity (a merge of control).

A fork or merge is shown by a **synchronization bar**  a heavy line with one or more input arrows and one or more output arrows.



SOFTWARE ENGINEERING

Activity diagram example

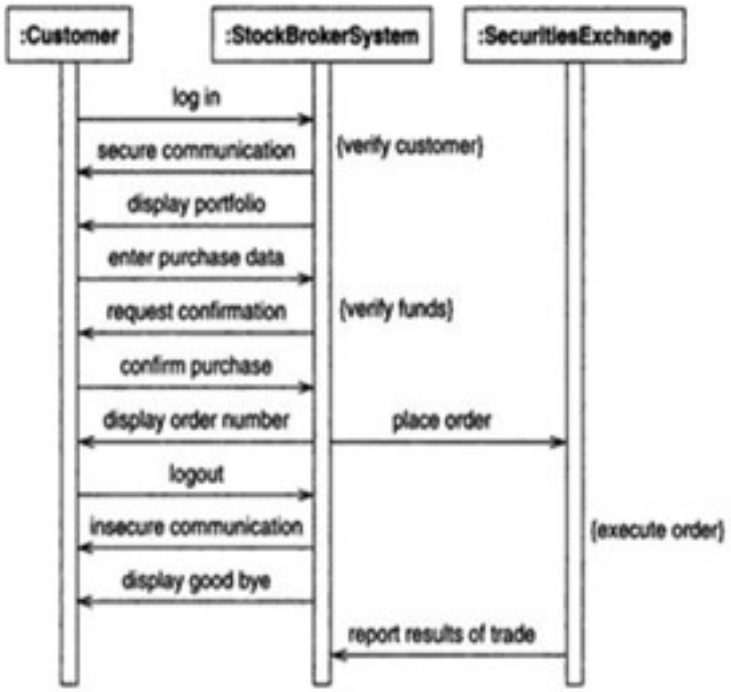


Figure 7.5 Sequence diagram for a session with an online stock broker. A sequence diagram shows the participants in an interaction and the sequence of messages among them.

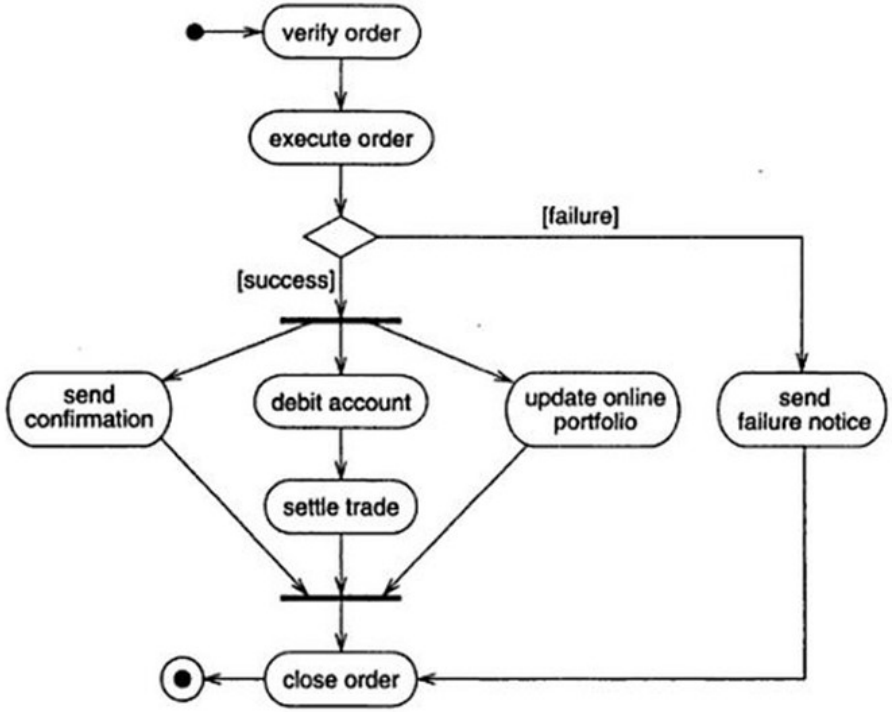


Figure Activity diagram for stock trade processing. An activity diagram shows the sequence of steps that make up a complex process.

SOFTWARE ENGINEERING

System Modeling and Design

Ms. Archana A, Sowmya BP & Akshatha PR
Computer Applications

SOFTWARE ENGINEERING

Activity Models

Archana A
Department of Computer Applications

Classroom Activity

1. Scenario: Online Shopping Process

Create a UML activity diagram for the process of online shopping. Include activities such as browsing products, adding items to the cart, entering shipping details, and completing the purchase.

2. Scenario: Library Book Borrowing System Design a UML activity diagram for the process of borrowing a book from a library. Include activities like searching for a book, checking book availability, borrowing the book, and returning the book if delay pay penalty.

3. Scenario: Travel Reservation System

Design a UML activity diagram for the process of making a travel reservation. Include activities like selecting the destination, choosing travel dates, entering passenger information, and confirming the reservation.

Classroom Activity

4. Scenario: Classroom Attendance System

Construct a UML activity diagram for a classroom attendance system. Include activities such as taking attendance, marking absent students, and updating the attendance records.

5. Scenario: Online Quiz Application

Create a UML activity diagram for an online quiz application. Include activities such as logging in, selecting a quiz, answering questions, and viewing the results.

THANK YOU

Ms. Archana A, Sowmya BP &Akshatha PR
Assistant Professor

Department of Computer Science
archana@pes.edu

+91 80 6666 3333 Extn 392